

如何使用

在 code 中 build 文件下使用 make 命令编译，编译成功后，在 build 文件夹下会生成可执行文件 main, 在 test 目录下的 main.json 调整初始条件即可使用不同的测试用例。

在文件 EquationSolverFactory.h 中，设计了工厂模式以及单例模式，为选择不同的求解器提供了方便。

```
1 {  
2     "FuncName" : "Func",  
3     "method" : "AB4",  
4     "output_file" : "output",  
5     "condition_type" : 1,  
6     "step" : 200000,  
7     "is_test" : false  
8 }
```

1. method 选择不同的方法与精度
2. conditiontype 选择不同的初始条件
3. step 选择不同的步长
4. is_test 选择是否进行测试

下面只贴出 AB4,AM5 和 BDF4 在步长为 200000 时的数值结果。

AB4 方法

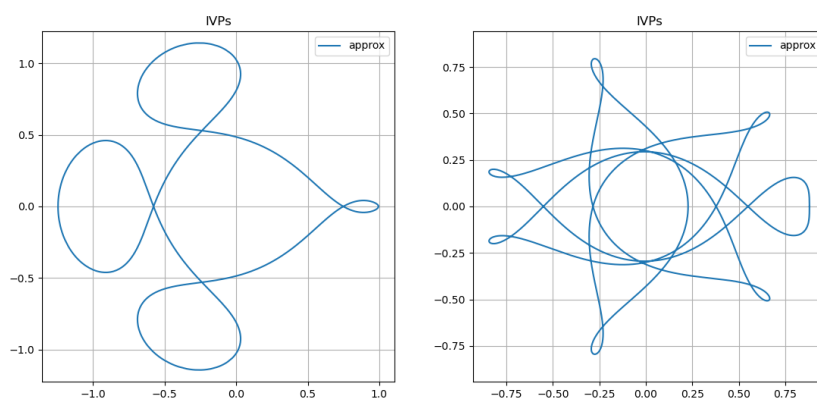


图 1: AB4 方法

AB4 最终误差 = 0.0170842

AB4 最终误差 = 2.20725e-06

AM5 方法

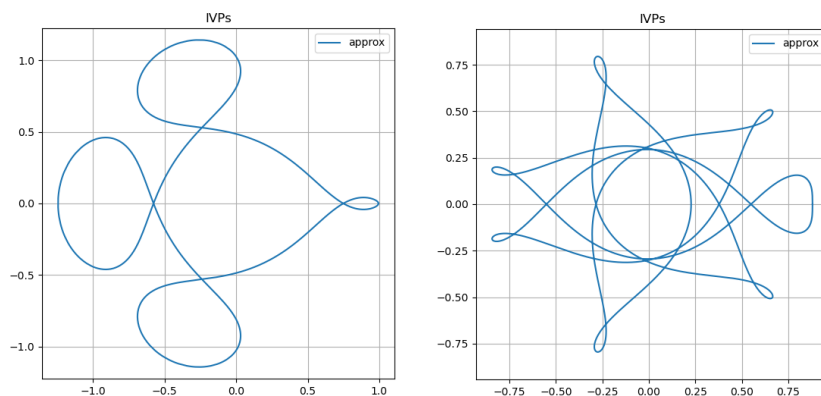


图 2: AM5 方法

AM5 最终误差 = $1.59796\text{e-}05$

AM5 最终误差 = $8.88581\text{e-}11$

BDF4 方法

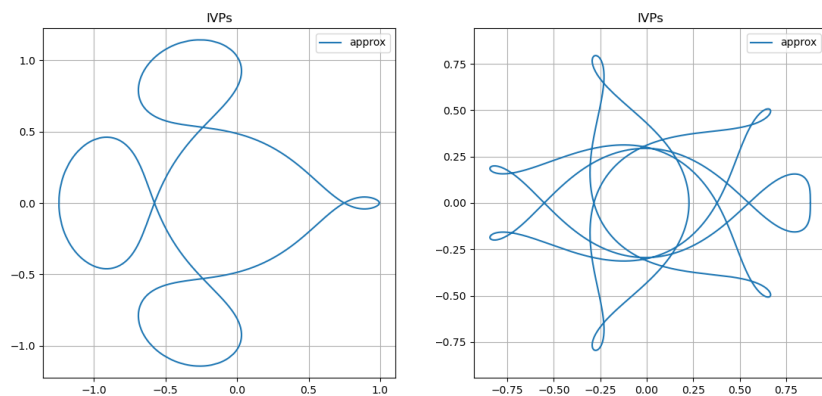


图 3: BDF4 方法

BDF4 最终误差 = 0.00974762

BDF4 最终误差 = $1.85899\text{e-}10$

收敛阶

对于一个 p 阶方法, 用步长 h 求解一个 IVPs 时, 其全局误差 $E(h)$ 应满足 $E(h) \approx O(h^p)$ 。对于两个不同的网格 $h_1 = \frac{T}{N_1}, h_2 = \frac{T}{N_2}$, 可以用式 $p \approx \frac{\log(E_1/E_2)}{\log(h_1/h_2)}$ 来估算收敛阶。

下面我们利用 (10.93) 式, 计算收敛阶。

表 1: AB4、BDF4 与 AM5 方法在不同步长下的误差与收敛阶

方法	N	步长 h	误差 $\ u - u_h\ $	收敛阶 p
AB4	10000	0.00191405	2.20725×10^{-6}	—
	20000	0.00095703	7.80855×10^{-8}	4.82
	40000	0.00047851	9.30889×10^{-9}	3.07
	80000	0.00023926	6.64014×10^{-10}	3.81
BDF4	10000	0.00191405	1.48016×10^{-6}	—
	20000	0.00095703	4.05638×10^{-8}	5.19
	40000	0.00047851	5.26787×10^{-9}	2.94
	80000	0.00023926	5.22306×10^{-10}	3.33
AM5	10000	0.00191405	1.93293×10^{-7}	—
	20000	0.00095703	6.11855×10^{-9}	4.98
	40000	0.00047851	2.64776×10^{-10}	4.53
	80000	0.00023926	9.28519×10^{-11}	1.51

发现 AB4 和 BDF4 的收敛阶都超过了 4, 这是因为初值给定的非常精确而出现了超收敛的现象。而 AM5 的收敛阶却突然减小, 原因是其误差已经达到了机器精度, 导致相对误差变得不再可靠。